



Project 1: Optimising High Performance Data Processing for Large-Scale Web Crawlers

MAUKERJA.MY

Job listing dataset

Group: TripleA

LAM YOKE YU A23CS0233

NABIL AFLAH BOO BINTI MOHD YOSUF BOO YONG CHONG A23CS0252

ANIS SAFIYYA BINTI JANAI A23CS0049

INTRODUCTION

Project Background

Online recruitment platforms like **Maukerja** accumulate massive job posting data. Traditional data processing faces bottlenecks in execution time, memory, and scalability. This project builds a high-performance pipeline comparing **Pandas, Polars, and PySpark** on **106,081 unique job records** crawled from Maukerja.

Target

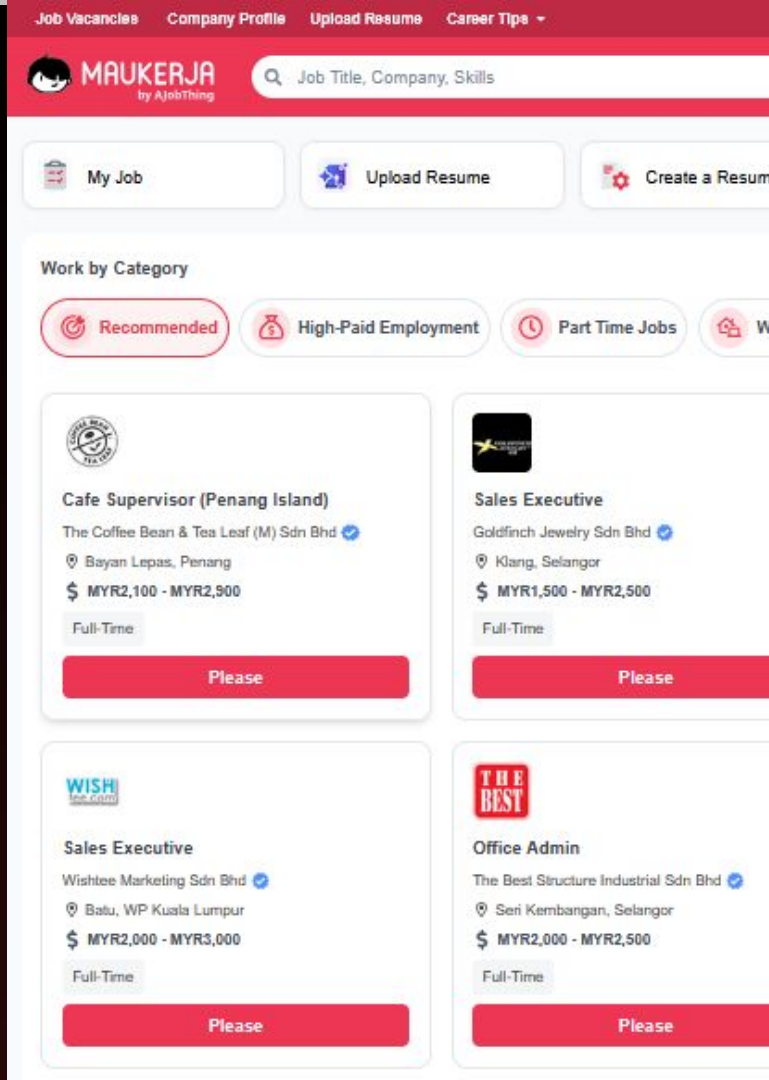
Maukerja : Malaysia & Singapore
recruitment platform

Dataset

106,081 unique job records
collected

Frameworks

Pandas, Polars & PySpark benchmarked



Objectives

1 Automated Web Crawler

Systematically collect job posting data from Maukerja using BeautifulSoup & Requests.

3 Framework Benchmarking

Evaluate Pandas, Polars, and PySpark using execution time, CPU, memory, and throughput.

2 Data Preprocessing

Clean and standardise datasets for quality, consistency, and relevance.

4 Optimisation Strategies

Identify the most efficient framework and demonstrate effectiveness of optimisation techniques.

Performance Bottlenecks



Core Challenges

Traditional data processing struggles with large-scale web scraper datasets:

Execution Time

Slow processing limits analytical efficiency at scale.

Memory Usage

High RAM consumption restricts dataset size.

Scalability

Traditional methods fail to handle 100K+ records effectively.

System Architecture

Web Scraping

BeautifulSoup,
Requests, Regex



Benchmarking

Pandas, Polars, PySpark



Data Processing

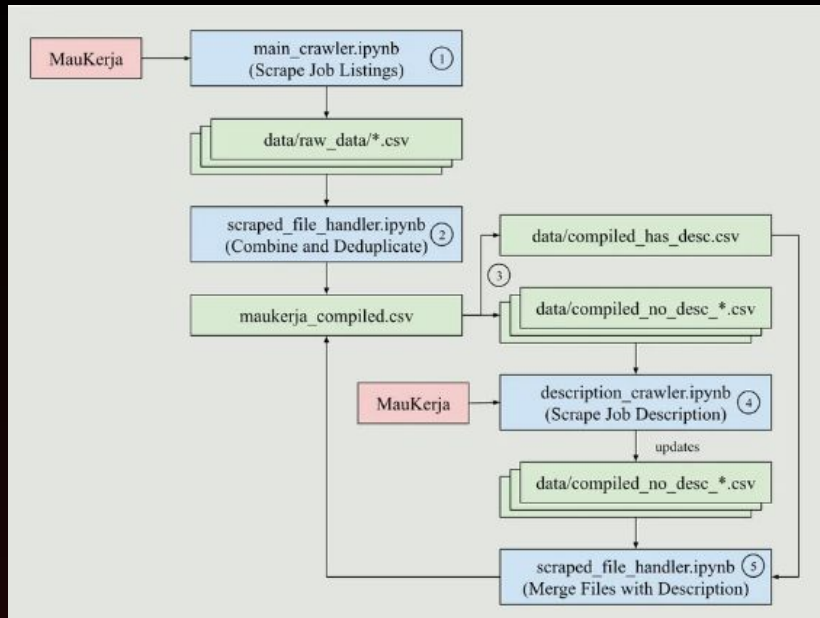
Deduplication,
cleaning,
transformation



Visualisation

Matplotlib, Seaborn





1,106

CSV Files

Generated from filter combinations

106,081

Unique Records

After deduplication

175h

Total Runtime

~50h listings + ~125h descriptions

Data Collection

1

Step 1

Job listing page crawling

2

Step 2

Combine scraped files

3

Step 3

File separation

4

Step 4

Scrape job descriptions

5

Step 5

Dataset recombination

6

Step 6

Repeat

Data Processing

Cleaning Methods

- Deduplication by unique job ID
- Salary normalisation to unified numerical format
- Standardisation of state/location fields
- Removal of irrelevant columns & missing records
- Column renaming for consistent analytical format

Transformation & Storage

- Salary split into `salary_min` / `salary_max` columns
- Categorical encoding for low-cardinality string columns
- CSV + GZIP compression for disk efficiency
- Vectorised operations preferred over row-wise iteration

Four Methods Compared



Pandas Baseline

Reads all columns with runtime type inference that is row-wise and using `.apply()` for salary classification.



Pandas Optimised

Column pruning, explicit dtypes, categorical encoding, vectorised `pd.cut()` binning.



Polars

API with query plan optimisation, multithreaded Rust engine, and automatic parallelism.



PySpark

Local mode Spark with predefined schema and in-memory caching

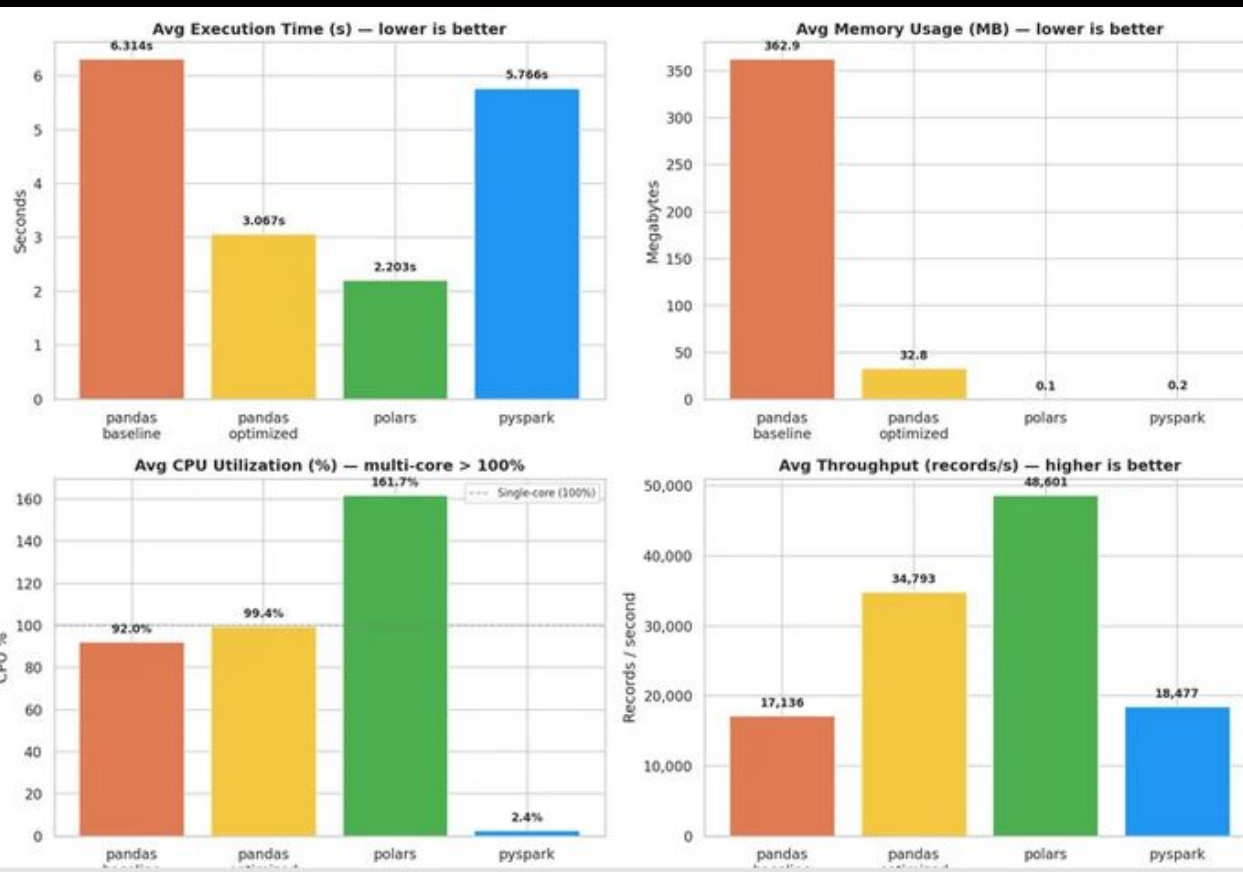
Comparison

There are 4 key performance metrics that have been measures across libraries, including:

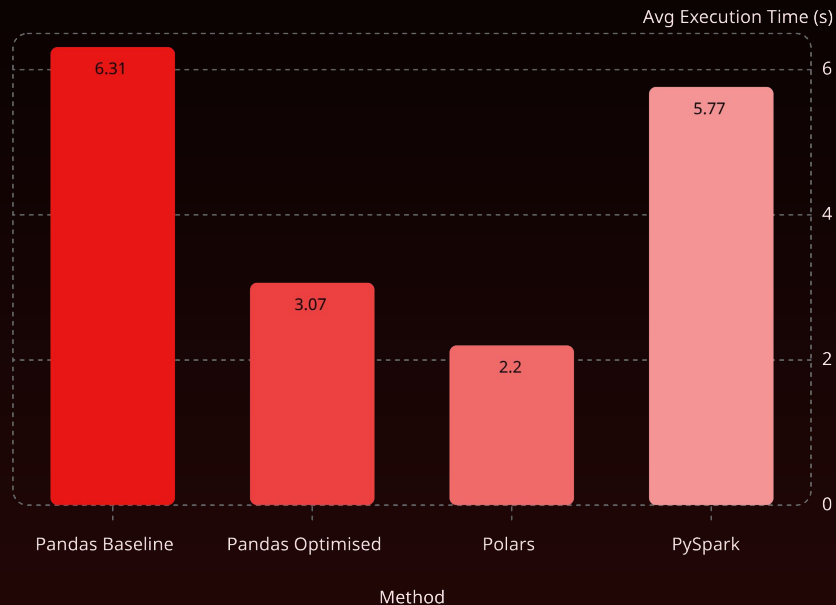
- Execution time (s)
- Memory usage (MB)
- CPU utilization (%)
- Throughput (records/sec)

		time_sec	cpu_%	memory_mb	throughput	avg_time_sec	avg_cpu_%	avg_memory_mb	avg_throughput	speedup_vs_baseline
method	run									
pandas_baseline	1	7.5852s	78.0%	362.89	13,985					
	2	5.8972s	98.7%	362.89	17,988	6.3135s	92.0%	362.89	17,136	1.00x
	3	5.4582s	99.4%	362.90	19,435					
pandas_optimized	1	2.8967s	100.1%	32.82	36,621					
	2	3.4061s	98.5%	32.82	31,144	3.0667s	99.4%	32.82	34,793	2.06x
	3	2.8973s	99.6%	32.82	36,614					
polars	1	1.9205s	185.3%	0.08	55,236					
	2	2.3888s	185.4%	0.04	44,407	2.2025s	161.7%	0.06	48,601	2.87x
	3	2.2982s	114.4%	0.04	46,159					
pyspark	1	5.5027s	2.5%	0.22	19,278					
	2	6.3144s	2.2%	0.22	16,800	5.7660s	2.4%	0.22	18,477	1.09x
	3	5.4810s	2.6%	0.21	19,354					

Visualization



Performance Evaluation



Key Findings



Polars – Best Overall

2.87× speedup, 48,601 rec/s throughput, ~0.1 MB memory, 161.7% CPU (multi-core).

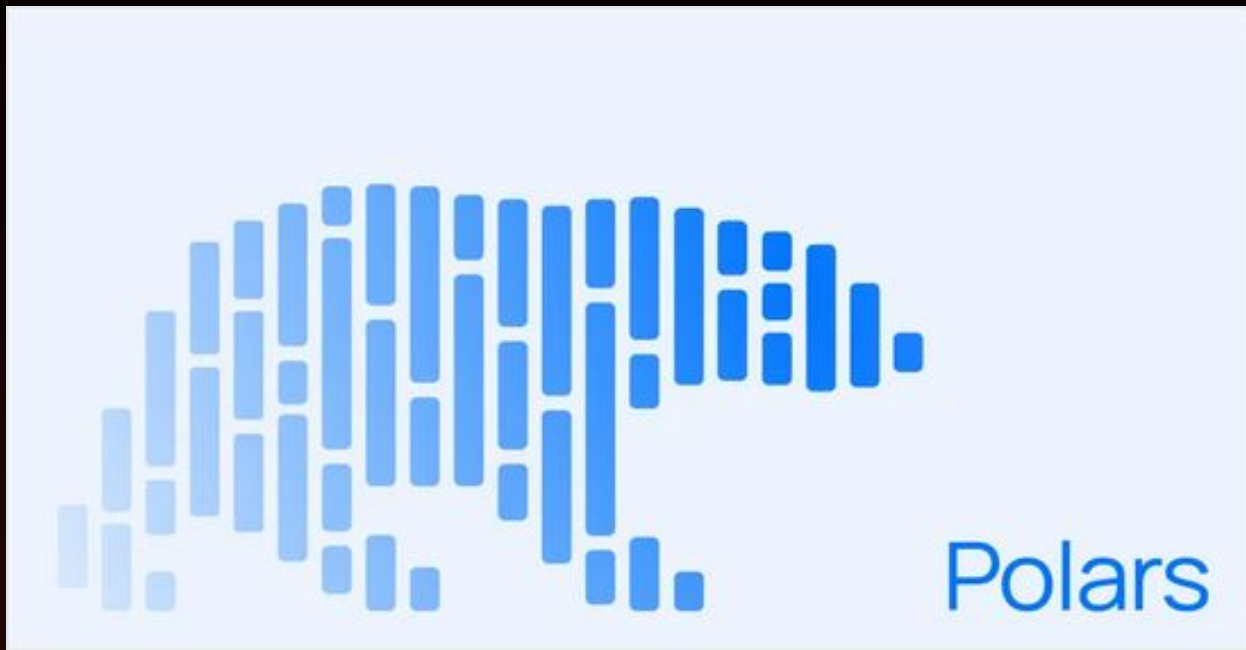
Pandas Optimised

2.06× speedup, memory reduced from 362.9 → 32.8 MB via column pruning & categorical encoding.

PySpark

Only **1.09× speedup** with JVM overhead dominates at this dataset scale on a single machine.

Winner 🏆



Challenges & Limitations

Challenges encountered

- Regular expression issues
- Termination logic issues
- Insufficient data volume for the benchmarking phase
- Increase processing time as the file size grows

Limitations of the solution

- Heavily dependent on human intervention
- Hard-coded logic in data cleaning
- Limited benchmarking attributes

Summary & Future Work

What was done:

- Data Collection – Web Scraping
- Data Cleaning
- Benchmarking

Results:

- Best overall performance: Polars

Future Work

- Extend to handle larger datasets
- Include other data processing framework in the comparison

THANK YOU

For your attention